

Thirteen numbers, on paper and then in NumPy

The 2–3–1 network from slide 21, one example through it. Compute every number with a pen first, then write the forward pass in three lines and check the two agree digit by digit.

Guided lab

18 minutes

Paper required

9 weights · 4 biases

Before you start Your network · given 1 · the whole pass by hand 2 · the forward pass in NumPy

3 · assert that they agree 4 · name every number 5 · the network from Friday Reflection

If you finish early

Before you start

Open the provided `lab09.ipynb`. The numbers are already in the first cell — Lab 5's layer, with a second layer added.



By the end of this lab you can

- Hand-compute a full 2–3–1 forward pass, every intermediate shown.
- Write the forward pass as three lines of NumPy.
- Assert that paper and code agree, and name every number in the network.

LAB 9

Your network · given

The first layer is **Lab 5's layer, unchanged** — you already know its pre-activations are `[3.5, -1.5, -3.0]`. What is new is the sigmoid after it and the second layer.

given

PYTHON

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

X = np.array([[2.0, -1.0]])           # (1, 2)
W1 = np.array([[ 1.0, 0.5, -2.0],
               [-1.0, 2.0, 0.0]])     # (2, 3)
b1 = np.array([0.5, -0.5, 1.0])      # (3,)
W2 = np.array([[1.0], [-2.0], [0.5]]) # (3, 1)
b2 = np.array([0.2])                 # (1,)
```

TASK 1 · 6 MINUTES · PAPER FIRST

the whole pass by hand

1

A calculator is fine for the sigmoid. Keep **three decimals** throughout.

PYTHON

```
Z1 = [ 3.5 , -1.5 , -3.0 ]           ← from Lab 5
A1 = [ σ(3.5) , σ(-1.5) , σ(-3.0) ] = [ __ , __ , __ ]
Z2 = 1.0·A1[0] + (-2.0)·A1[1] + 0.5·A1[2] + 0.2 = __
A2 = σ(Z2) = __
```



$\sigma(z) = 1 / (1 + e^{(-z)})$. A negative score gives a number **below** 0.5 — if $\sigma(-1.5)$ came out above 0.5, check the sign in the exponent.

your turn

PYTHON

```
paper_A1 = [None, None, None]    # TODO: your three hand-computed activations
paper_Z2 = None                  # TODO
paper_A2 = None                  # TODO
```

one version that works

PYTHON

```
paper_A1 = [0.971, 0.182, 0.047]
paper_Z2 = 0.830
paper_A2 = 0.696
```

TASK 2 · 4 MINUTES

the forward pass in NumPy

2

Three lines inside the function. Each is a layer or an activation; none needs a loop.

your turn

PYTHON

```
def forward(X, W1, b1, W2, b2):
    Z1 = X @ W1 + b1
    A1 = None          # TODO
    Z2 = None          # TODO
    A2 = None          # TODO
    return Z1, A1, Z2, A2

Z1, A1, Z2, A2 = forward(X, W1, b1, W2, b2)
print("Z1", np.round(Z1, 3)); print("A1", np.round(A1, 3)); print("Z2", np.round(Z2, 3));
print("A2", np.round(A2, 3))
```

what it should print

OUTPUT

```
Z1 [[ 3.5 -1.5 -3. ]]
A1 [[0.971 0.182 0.047]]
Z2 [[0.83]]
A2 [[0.696]]
```

TASK 3 · 2 MINUTES

assert that they agree

3

If this fails, **do not change the paper to match**. Find which of the two is wrong.

given

PYTHON

```
assert np.allclose(A1, [paper_A1], atol=2e-3)
assert np.allclose(Z2, paper_Z2, atol=2e-3)
assert np.allclose(A2, paper_A2, atol=2e-3)
print("paper and code agree")
```

what it should print

OUTPUT

paper and code agree

TASK 4 · 3 MINUTES

name every number

4

Fill in the shapes, then count. Nine weights and four biases is the network on slide 21.

your turn

PYTHON

```
shapes = {"X": None, "W1": None, "b1": None, "A1": None, "W2": None, "A2": None} # TODO:
tuples
n_weights = None # TODO: from the arrays, not typed in
n_biases = None # TODO

assert shapes == {k: v.shape for k, v in dict(X=X, W1=W1, b1=b1, A1=A1, W2=W2,
A2=A2).items()}
assert (n_weights, n_biases) == (9, 4)
print("every number accounted for")
```

one version that works

PYTHON

```
shapes = {"X": (1, 2), "W1": (2, 3), "b1": (3,), "A1": (1, 3), "W2": (3, 1), "A2": (1, 1)}
n_weights = W1.size + W2.size
n_biases = b1.size + b2.size
assert shapes == {k: v.shape for k, v in dict(X=X, W1=W1, b1=b1, A1=A1, W2=W2, A2=A2).items()}
assert (n_weights, n_biases) == (9, 4)
print("every number accounted for")
```

what it should print

OUTPUT

every number accounted for

TASK 5 · 3 MINUTES

the network from Friday

5

Your `forward` works on a batch without changing a character. Run it on the four XOR points with the hand-set weights from session 12's demonstration.

given

PYTHON

```
Xx = np.array([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
W1x = np.array([[4., 4., -4.], [4., -4., 4.]])
b1x = np.array([-2., 2., 2.])
W2x = np.array([[6.], [-6.], [-6.]])
b2x = np.array([3.])

_, _, _, out = forward(Xx, W1x, b1x, W2x, b2x)
print(np.round(out.ravel(), 3))      # XOR is 0, 1, 1, 0
```

what it should print

OUTPUT

[0.001 0.83 0.83 0.17]

Reflection



Write it in the notebook, in your own words

Point at `W1[0, 2]` in the given cell. In one sentence each: which input does it multiply, which hidden unit does it feed, and what would change in `A2` if it doubled?

Your answer:

If you finish early

A. Plot each hidden unit of the XOR network as a contour map over the square from -0.3 to 1.3 (session 12's step 5 shows how). Which unit is on for $(0, 1)$ but off for $(1, 1)$?

B. Change one number in `W2x` so the network computes **OR** instead of XOR. Check all four outputs.

your turn

PYTHON

```
# TODO (optional extensions A, B)
```